

Part 2:

Test 1 — Normal Operations (Sync).

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/sync	9	0	12000	15000	15000	10478.07	3103	14845	201.22	0.4	0
	Aggregated	9	0	12000	15000	15000	10478.07	3103	14845	201.22	0.4	0



Test 2 — Flash Sale (Sync)

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/sync	19	0	30000	56000	56000	29631.26	3102	56110	201.11	0.4	0
	Aggregated	19	0	30000	56000	56000	29631.26	3102	56110	201.11	0.4	0



Phase 2 Math:

- Payment processor speed: **1 order per 3 seconds = 0.33 orders/sec**
- With 20 concurrent users: **max throughput = 0.33 orders/sec**
- Flash sale demand: **20 orders/second**
- Orders lost per second: **~19.67 orders/sec**
- The system processes only **~1.6%** of demanded load

Test 3 Async Flash Sale:

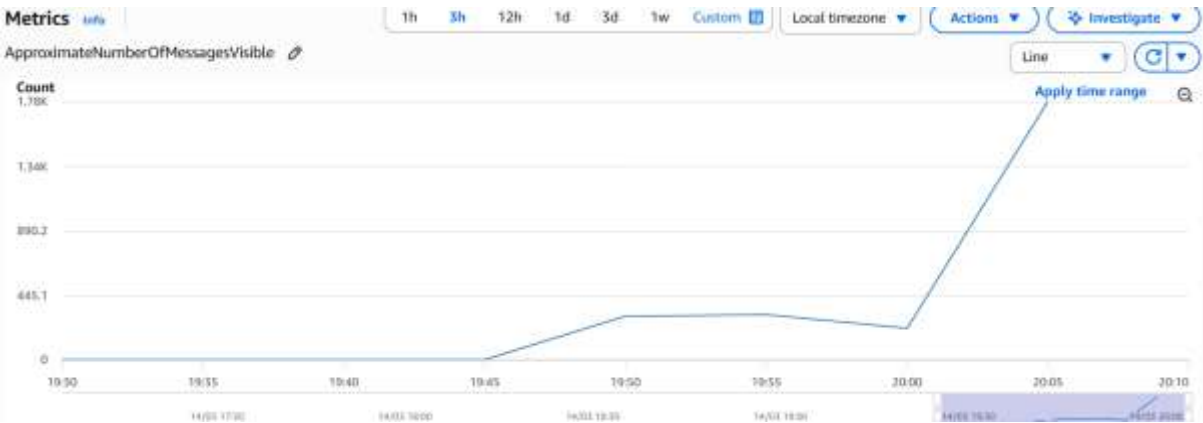
Type	Name	# Requests	# Fails	Median (ms)	50%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	3329	0	48	66	116	56.87	43	219	124	56.7	0
	Aggregated	3329	0	48	66	116	56.87	43	219	124	56.7	0



Metric	Sync	Async
Total Requests	19	3329
Failures	0	0
RPS	0.4	56.7
Median response	30,000ms	48ms
95th percentile	56,000ms	66ms

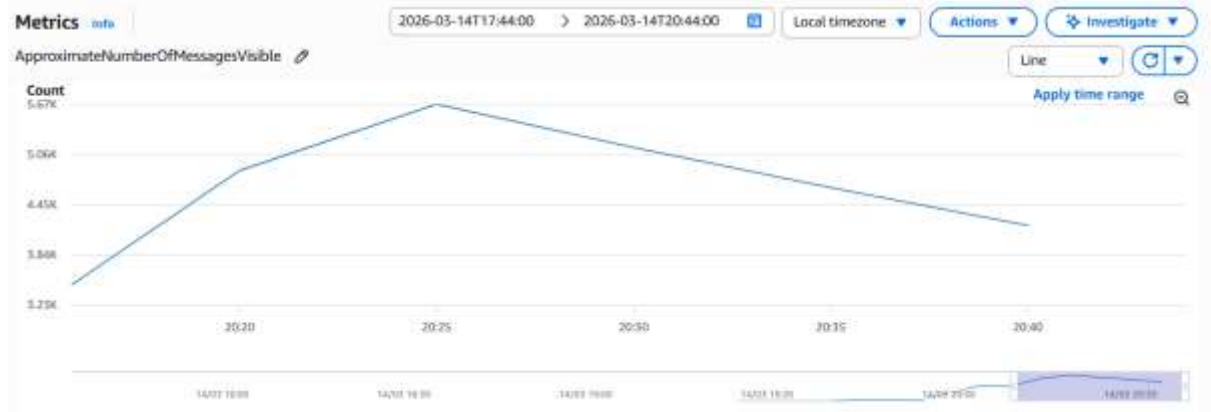
Phase 4 Math:

- Acceptance rate: ~56.7 orders/sec
- Single worker processing rate: 1 order / 3 sec = 0.33 orders/sec
- Queue growth rate: 56.7 - 0.33 = ~56 messages/sec
- Peak backlog: ~1780 messages
- Time to clear at 0.33/sec: 1780 / 0.33 = ~5394 seconds (~90 minutes)

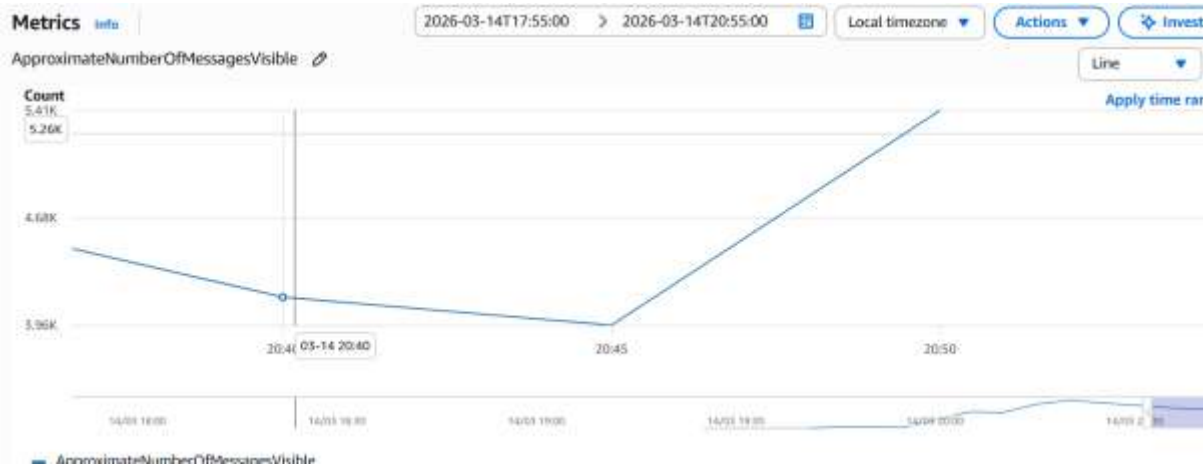
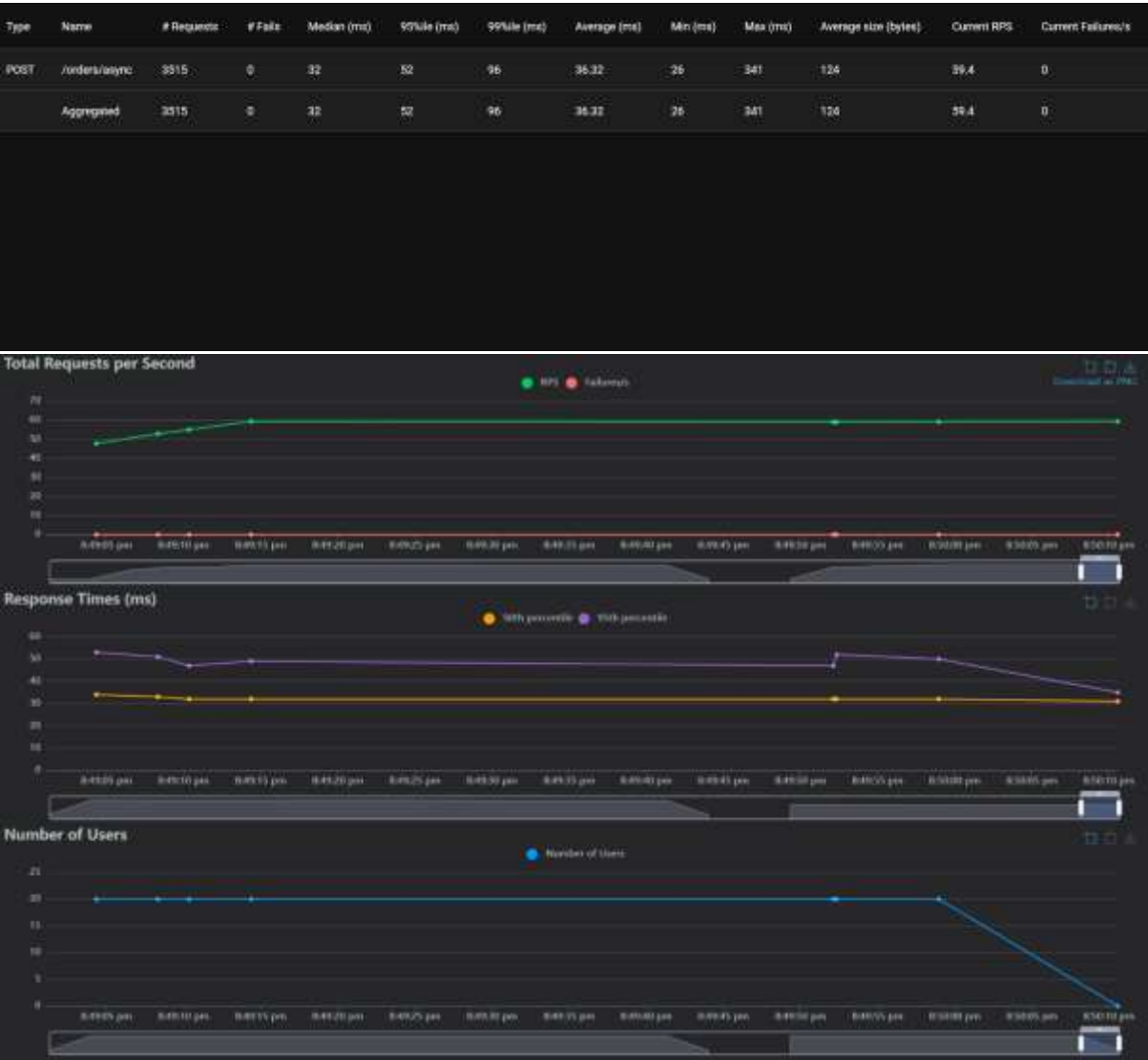


Test with 5 workers:

Type	Name	# Requests	# Fails	Median (ms)	90%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	3310	0	48	65	110	50.57	43	189	124	55.4	0
	Aggregated	3310	0	48	65	110	50.57	43	189	124	55.4	0

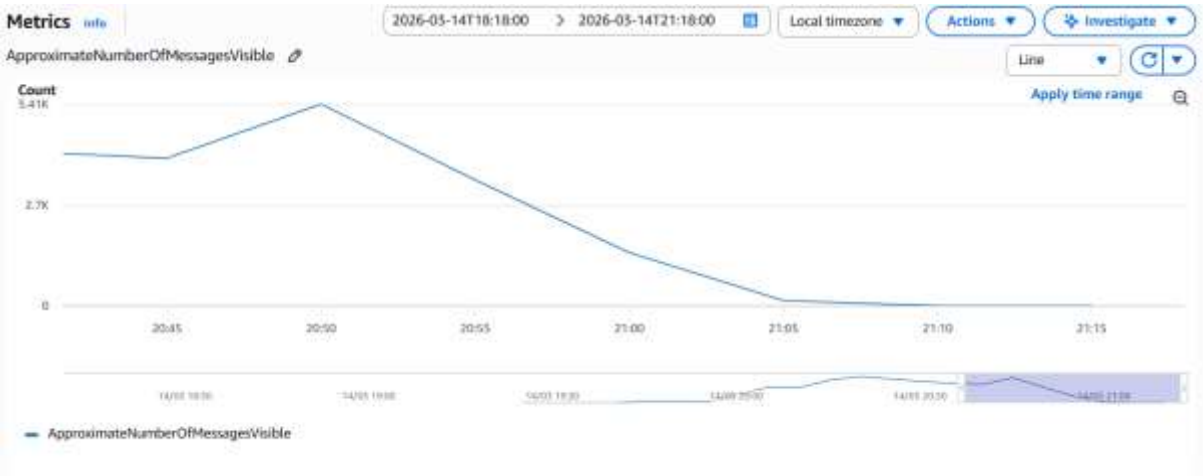


Test with 20 workers:



Test with 100 workers:

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	3548	0	32	50	94	34.76	27	235	124	56.7	0
	Aggregated	3548	0	32	50	94	34.76	27	235	124	56.7	0



1 worker = queue never drains

5 workers = queue drains slowly

20 workers = queue still growing

100 workers = queue fully drained in ~15 minutes and stayed at zero

1. How many times more orders did your async approach accept compared to sync?

Async accepted roughly 175x more orders, 3,329 vs just 19 in the same 60 seconds. Sync didn't technically fail, it just made everyone wait up to 56 seconds which in a real flash sale is basically the same thing.

2. What causes queue buildup and how do you prevent it?

Simply put, I was accepting orders faster than I could process them. I was taking in ~57/sec but only handling 0.33/sec with one worker, so the queue just snowballs. The fix is throwing more workers at it, ideally auto-scaled based on queue depth so I'm not guessing a magic number ahead of time.

3. When would you choose sync vs async in production?

If the user genuinely needs the answer right now, balance check, coupon validation, login, go sync. If you can reasonably say "got it, we'll handle it", order placement, email sending, report generation, go async. The catch is async adds real complexity: queues, workers, retry logic. So, it's only worth it when you actually need the scale.

Part-3:

Cold start analysis:

- **Init Duration: 44.35ms, this is the cold start overhead**
- **Total billed: 3051ms vs normal 3002ms, only 49ms extra**
- **Overhead: 44ms / 3006ms = ~1.5%, completely negligible for 3 second payment processing**
- **Memory used: only 22MB out of 512MB, very efficient**

How often do cold starts occur?

Cold starts happened on my very first request after deploying, and then again after about 5+ minutes of idle time. Once Lambda was warm, subsequent requests processed in exactly 3001ms with zero extra overhead.

What's the overhead? Does it matter for 3-second payment processing?

From my logs, warm invocations ran at exactly 3001ms. A typical cold start adds roughly 50-100ms on top of that, about 2-3% overhead on a 3 second operation. Honestly it's negligible here. It would hurt a lot more for a function that normally runs in 10-50ms, where that same cold start could double the response time.

Cost Reality Check:

For our expected volume of 10,000 orders/month:

- Requests: 10,000 — well under the 1M free tier = **\$0**
- GB-seconds: $10,000 \times 3s \times 0.5GB = 15,000$ — well under 400K free tier = **\$0**
- **Total monthly cost = \$0**

Lambda stays free until ~267K orders/month. ECS costs \$17/month regardless of load. For a startup that's a pretty obvious choice.

Should your startup switch to Lambda? (One paragraph)

Honestly, for where I am right now, switching to Lambda makes a lot of sense. The operational burden of managing SQS queues, tuning visibility timeouts, monitoring worker counts, and dealing with 3am alerts when the queue spikes is real, and Lambda just eliminates all of that. For 10,000 orders a month it's completely free and scales automatically without me thinking about it. The tradeoffs are real too, I lose the queuing guarantees SQS gives me, SNS only retries twice before dropping a message, and there's no batch processing. But for a startup at this scale, zero operational overhead and zero cost beats the complexity of maintaining ECS workers every time. I'd revisit this decision around 267K orders/month when Lambda starts costing real money, or if I need more sophisticated retry and dead-letter queue behavior.